	Politechnika Bydgoska im. J. J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki		
Przedmiot	Zaawansowane programowanie obiektowe		
Prowadzący	dr inż. Damian Szczegielniak		
Temat	Implementacja aplikacji bazodanowej przy użyciu JDBC		
Student	Jakub Ziółkowski, grupa 1, Informatyka stosowana, IV semestr, studia stacjonarne		
Nr ćw.	3	Data wykonania	16.05.2026 r.
Ocena		Data oddania spr.	12.07.2026 r.

Spis treści

1. Cel laboratorium.....	2
2. Rozwiązanie.....	3
2.1. Ogólny opis aplikacji.....	3
2.2. Struktura projektu i podział na pakiety	3
2.3. Klasy Projekt i Zadanie	6
2.4. Konfiguracja połączenia - klasa DataSource i wzorzec Singleton	8
2.5. Inicjalizacja bazy danych oraz transakcje w klasie DbInitializer	9
2.6. Warstwa dostępu do danych - interfejsy i implementacje DAO (ProjektDAO, ProjektDAOImpl, ZadanieDAO, ZadanieDAOImpl)	11
2.7. Konfiguracja mechanizmu rejestracji w pliku logback.xml	12
2.8. Graficzny interfejs użytkownika - plik FXML, Scene Builder i styl CSS.....	13
2.9. Kontroler aplikacji, czyli klasa ProjectController, ObservableList i wielowątkowość	14

2.10. Operacje CRUD w interfejsie - dodawanie, edycja i usuwanie projektów	15
2.11. Mechanizm wyszukiwania i stronicowania danych	17
2.12. Tworzenie wersji uruchomieniowej (jlink i jpackage)	17
2.13. Realizacja zadania dodatkowego.....	18
3. Wnioski końcowe/Podsumowanie	20

1. Cel laboratorium

Głównym celem tego laboratorium była implementacja okienkowej aplikacji bazodanowej w języku Java. Realizacja tego celu była możliwa dzięki zastosowaniu niskopoziomowego interfejsu dostępu do danych JDBC, mechanizmów optymalizacji operacji na bazie danych przy użyciu buforowania połączeń w technologii HikariCP, a także budowie graficznego interfejsu użytkownika przy użyciu technologii JavaFX i narzędzia Scene Builder.

W ramach zadania podzieliłem odpowiedzialność aplikacji na warstwy logiczne (Model-View-Controller) oraz zastosowałem wzorce projektowe, takie jak Singleton oraz Data Access Object (DAO). Zastosowałem też przetwarzanie wielowątkowe oraz asynchronicznie oddelegowałem zadania wejścia/wyjścia (zapytania SQL do bazy danych) do wątków roboczych zarządzanych przez ExecutorService.

Ponadto, laboratorium stawiało za cel opanowanie wdrażania i dystrybucji oprogramowania Java z wykorzystaniem systemu modułów (JPMS) oraz narzędzi jlink i jpackage do budowania pakietów instalacyjnych aplikacji.

2. Rozwiązanie

2.1. Ogólny opis aplikacji

Laboratorium zakładało przygotowanie aplikacji desktopowej służącej do wyszukiwania, modyfikowania oraz usuwania projektów oraz przypisanych do nich zadań. Wbudowana relacyjna baza danych HSQLDB (HyperSQL Database), działa w trybie plikowym, gwarantując trwałość danych pomiędzy kolejnymi uruchomieniami programu. Warstwa prezentacji została oddzielona od warstwy danych poprzez zastosowanie kodu FXML, w którym zdefiniowałem rozmieszczenie elementów graficznych, przycisków sterujących, pól tekstowych oraz tabeli głównej.

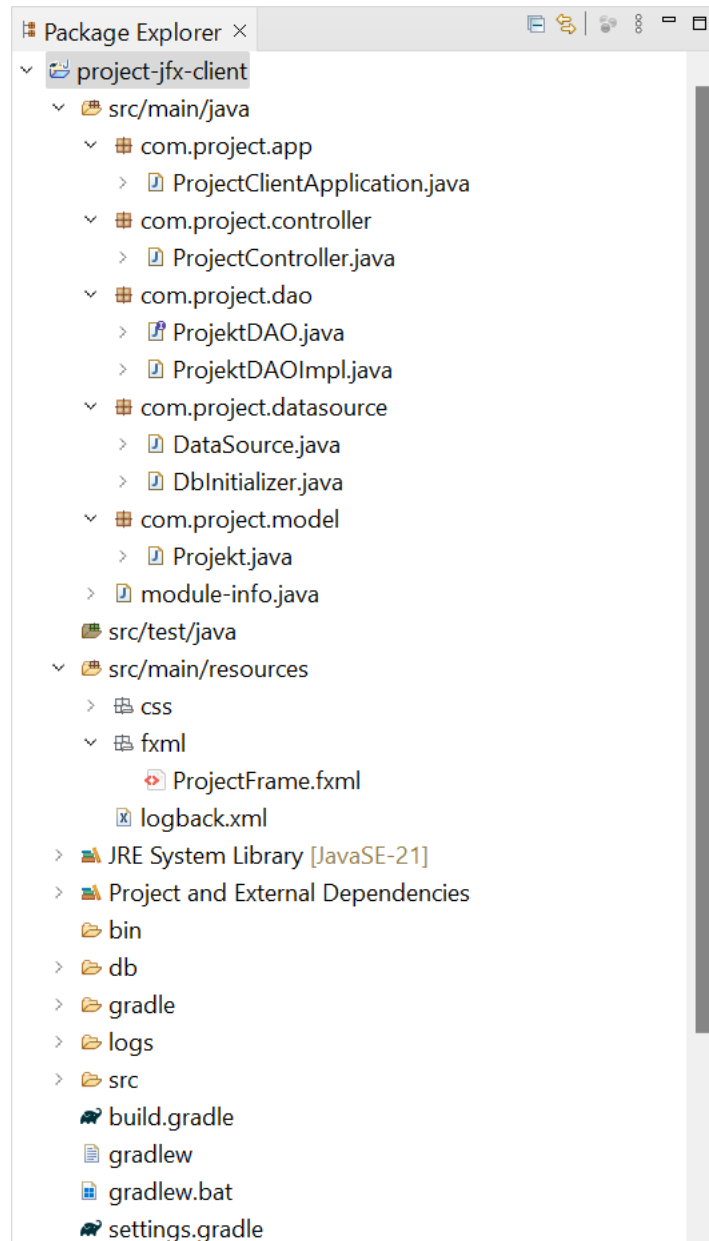
Program umożliwia pełną interakcję z bazą danych w czasie rzeczywistym. Dane tabelaryczne są automatycznie synchronizowane z listami JavaFX (typu `ObservableList`), co sprawia, że każda poprawnie zatwierdzona w bazie danych modyfikacja (dodanie wiersza, aktualizacja pól, usunięcie rekordu) jest natychmiast odzwierciedlana na ekranie bez konieczności ponownego przeładowywania całego widoku. Aplikacja korzysta z dynamicznego stronicowania wyników, co pozwala na płynne przeglądanie nawet tysięcy rekordów, a także posiada wbudowany zaawansowany parser wyrażeń regularnych, który automatycznie określa kryteria zapytania SQL na podstawie formatu wprowadzonej w polu wyszukiwania frazy.

2.2. Struktura projektu i podział na pakiety

Kod źródłowy został oparty o modularny projekt Gradle. Poniżej znajduje się opis struktury pakietów:

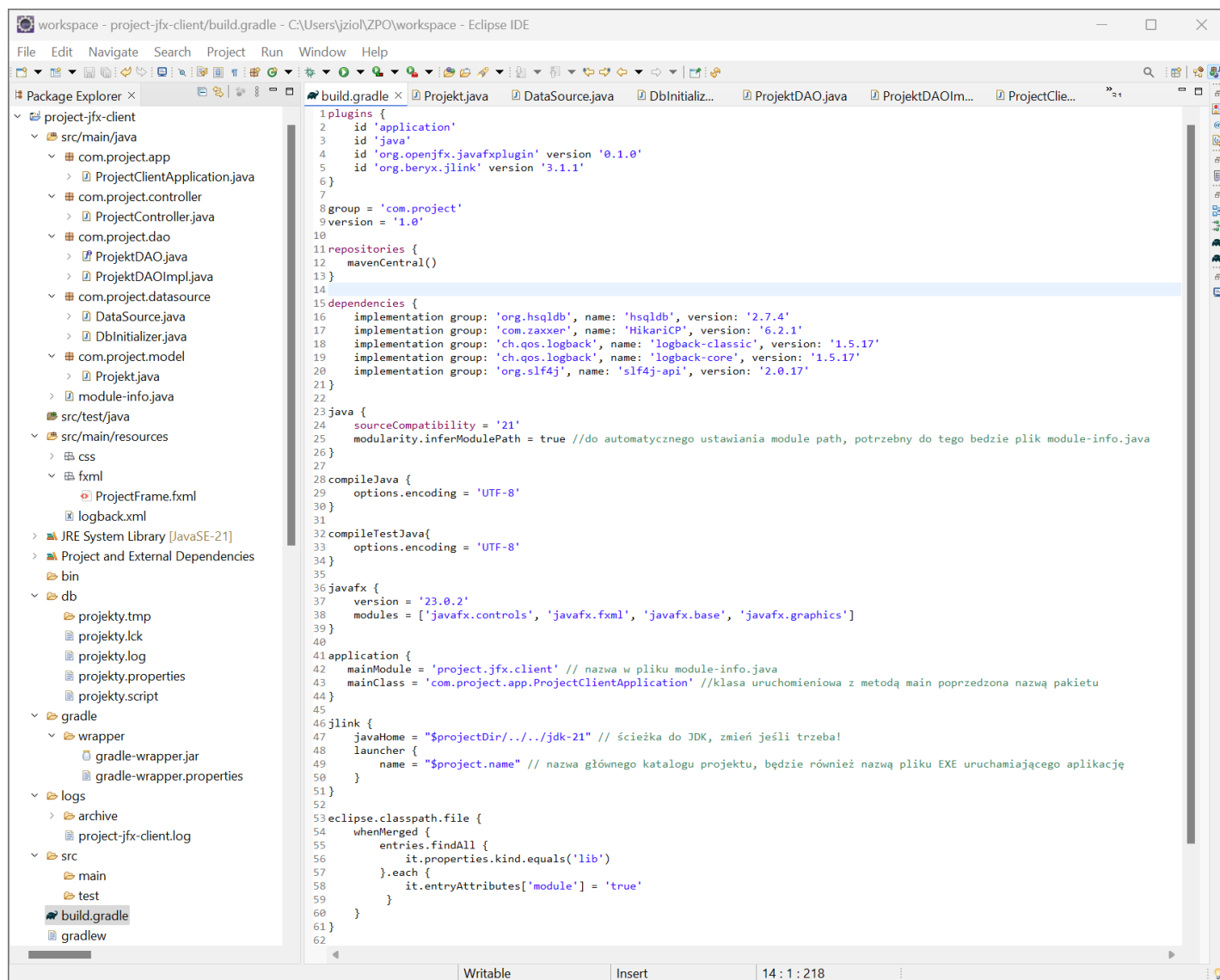
1. `com.project.model` - zawiera klasy domenowe (POJO), które odzwierciedlają obiektowo tabele relacyjne (klasa `Projekt` i dla zadania dodatkowego klasa `Zadanie`).
2. `com.project.datasource` - pakiet odpowiedzialny za infrastrukturę bazodanową niskiego poziomu. Zawiera implementację puli połączeń oraz automatyczny inicjalizator schematu struktur.
3. `com.project.dao` - grupuje interfejsy definiujące abstrakcyjne operacje CRUD oraz ich konkretne implementacje wykonujące zapytania SQL przez JDBC.
4. `com.project.controller` - zarządza logiką biznesową interfejsu użytkownika, obsługą zdarzeń, asynchronicznym przetwarzaniem wątków oraz walidacją danych.
5. `com.project.app` - zawiera główną klasę rozruchową, która inicjalizuje framework JavaFX i łączy zasoby XML.

Zasoby takie jak pliki widoków (.fxml), arkusze stylów (.css) oraz plik konfiguracyjny mechanizmu rejestracji zdarzeń (logback.xml) są umieszczone w dedykowanym katalogu zasobów src/main/resources.



Screen 1 - struktura pakietów w widoku Package Explorer dla projektu podstawowego

Konfiguracja automatyzacji budowania opiera się na pliku build.gradle. Zostały w nim zdefiniowane wtyczki systemowe, wersja Javy (zgodność z JDK 21) oraz zestaw niezbędnych bibliotek zewnętrznych pobieranych automatycznie z repozytorium Maven Central. Do najważniejszych zależności należą: org.hsqldb:hsqldb (silnik bazy danych), com.zaxxer:HikariCP (zarządzanie połączeniami), slf4j-api oraz ch.qos.logback dla logowania operacji.



The screenshot shows the Eclipse IDE interface. On the left, the Package Explorer displays the project structure for 'project-jfx-client'. The main editor window shows the 'build.gradle' file with the following content:

```
1 plugins {
2   id 'application'
3   id 'java'
4   id 'org.openjfx.javafxplugin' version '0.1.0'
5   id 'org.beryx.jlink' version '3.1.1'
6 }
7
8 group = 'com.project'
9 version = '1.0'
10
11 repositories {
12   mavenCentral()
13 }
14
15 dependencies {
16   implementation group: 'org.hsqldb', name: 'hsqldb', version: '2.7.4'
17   implementation group: 'com.zaxxer', name: 'HikariCP', version: '6.2.1'
18   implementation group: 'ch.qos.logback', name: 'logback-classic', version: '1.5.17'
19   implementation group: 'ch.qos.logback', name: 'logback-core', version: '1.5.17'
20   implementation group: 'org.slf4j', name: 'slf4j-api', version: '2.0.17'
21 }
22
23 java {
24   sourceCompatibility = '21'
25   modularity.inferModulePath = true //do automatycznego ustawiania module path, potrzebny do tego bedzie plik module-info.java
26 }
27
28 compileJava {
29   options.encoding = 'UTF-8'
30 }
31
32 compileTestJava {
33   options.encoding = 'UTF-8'
34 }
35
36 javafx {
37   version = '23.0.2'
38   modules = ['javafx.controls', 'javafx.fxml', 'javafx.base', 'javafx.graphics']
39 }
40
41 application {
42   mainModule = 'project.jfx.client' // nazwa w pliku module-info.java
43   mainClass = 'com.project.app.ProjectClientApplication' //klasa uruchomieniowa z metodą main poprzedzona nazwą pakietu
44 }
45
46 jlink {
47   javaHome = "${projectDir}/../jdk-21" // ścieżka do JDK, zmień jeśli trzeba!
48   launcher {
49     name = "${project.name}" // nazwa głównego katalogu projektu, będzie również nazwą pliku EXE uruchamiającego aplikację
50   }
51 }
52
53 eclipse.classpath.file {
54   whenMerged {
55     entries.findAll {
56       it.properties.kind.equals('lib')
57     }.each {
58       it.entryAttributes['module'] = 'true'
59     }
60   }
61 }
62
```

Screen 2 - zawartość pliku budującego build.gradle

2.3. Klasy Projekt i Zadanie

Klasy modelowe Projekt i Zadanie to podstawowe elementy wymiany danych w aplikacji. Klasa Projekt przechowuje pięć zmiennych prywatnych: unikalny identyfikator projektId (Integer), nazwę projektu nazwa (String), opis (String), znacznik czasu utworzenia dataCzasUtworzenia (LocalDateTime) oraz planowaną datę oddania dataOddania (LocalDate). Nowoczesne typy z pakietu java.time wykorzystane do mapowania dat upraszczają operacje kalendarzowe i zapewniają integrację ze sterownikami JDBC.

W ramach zadania dodatkowego zaimplementowałem również klasę modelową Zadanie. Zawiera ona pola: zadanieId (Integer), nazwa (String), opis (String), kolejnosc (Integer), dataCzasUtworzenia (LocalDateTime) oraz klucz obcy projektId (Integer), wskazujący relację przynależności do konkretnego projektu. Obie klasy posiadają pełen zestaw akcesorów i mutatorów, konstruktory bezparametrowe oraz konstruktory sparametryzowane.

```
Projekt.java ×
1 package com.project.model;
2
3 import java.time.LocalDate;
4 import java.time.LocalDateTime;
5
6 public class Projekt {
7     private Integer projektId;
8     private String nazwa;
9     private String opis;
10    private LocalDateTime dataCzasUtworzenia;
11    private LocalDate dataOddania;
12
13    public Integer getProjektId() {
14        return projektId;
15    }
16    public void setProjektId(Integer projektId) {
17        this.projektId = projektId;
18    }
19    public String getNazwa() {
20        return nazwa;
21    }
22    public void setNazwa(String nazwa) {
23        this.nazwa = nazwa;
24    }
25    public String getOpis() {
26        return opis;
27    }
28    public void setOpis(String opis) {
29        this.opis = opis;
30    }
31    public LocalDateTime getDataCzasUtworzenia() {
32        return dataCzasUtworzenia;
33    }
34    public void setDataCzasUtworzenia(LocalDateTime dataCzasUtworzenia) {
35        this.dataCzasUtworzenia = dataCzasUtworzenia;
36    }
37    public LocalDate getDataOddania() {
38        return dataOddania;
39    }
40    public void setDataOddania(LocalDate dataOddania) {
41        this.dataOddania = dataOddania;
42    }
43
44    public Projekt() {
45        super();
46    }
47
48    public Projekt(Integer projektId, String nazwa, String opis, LocalDateTime dataCzasUtworzenia,
49        LocalDate dataOddania) {
50        super();
51        this.projektId = projektId;
52        this.nazwa = nazwa;
53        this.opis = opis;
54        this.dataCzasUtworzenia = dataCzasUtworzenia;
55        this.dataOddania = dataOddania;
56    }
57
58    public Projekt(String nazwa, String opis, LocalDateTime dataCzasUtworzenia, LocalDate dataOddania) {
59        super();
60        this.nazwa = nazwa;
61        this.opis = opis;
62        this.dataCzasUtworzenia = dataCzasUtworzenia;
63        this.dataOddania = dataOddania;
64    }
65
66    public Projekt(String nazwa, String opis, LocalDate dataOddania) {
67        this.nazwa = nazwa;
68        this.opis = opis;
69        this.dataOddania = dataOddania;
70    }
71 }
```

Screen 3 - kod źródłowy klasy Projekt.java z widocznymi konstruktorami i polami

2.4. Konfiguracja połączenia - klasa DataSource i wzorzec Singleton

W klasie `com.project.datasource.DataSource` zaimplementowałem mechanizm buforowania połączeń HikariCP, żeby zoptymalizować tworzenie nowego fizycznego połączenia z bazą danych dla każdego zapytania SQL. Klasa ta opiera się na wzorcu Singleton - posiada prywatny konstruktor, który zapobiega tworzeniu dodatkowych instancji, a cała konfiguracja jest ładowana jeden raz w bloku `static` klasy.

Pula połączeń opiera się o obiekt `HikariConfig`. Ustawiłem maksymalny rozmiar puli (`MaximumPoolSize`) na wartość 1, co jest wystarczające i zapobiega blokowaniu plików bazy danych HSQLDB przez inne procesy. Adres URL bazy danych jest generowany z uwzględnieniem parametrów:

- `sql.syntax_pgs=true` - aktywuje pełną zgodność ze składnią PostgreSQL (np. obsługa typu `SERIAL` czy `now()`),
- `hsqldb.write_delay=false` - eliminuje buforowanie zapisu w pamięci RAM, zabezpieczając dane przed uszkodzeniem w razie nagłego zamknięcia programu.

```
DataSource.java ×
1 package com.project.datasource;
2
3 import java.sql.Connection;
4 import java.sql.SQLException;
5 import com.zaxxer.hikari.HikariConfig;
6 import com.zaxxer.hikari.HikariDataSource;
7
8 public class DataSource {
9
10     private final static String DB_DIR = "db";
11     private final static String DB_NAME = "projekty";
12     private final static String DB_USERNAME = "admin";
13     private final static String DB_USER_PASSWORD = "admin";
14     /*
15     1. sql.syntax_pgs - this property, when set true, enables support for TEXT and SERIAL types.
16     It also enables NEXTVAL, CURRVAL and LASTVAL syntax and also allow compatibility with some other aspects of this dialect.
17     2. hsqldb.write_delay - If the property is true, the default WRITE DELAY property of the database is used, which is 500 milliseconds.
18     If the property is false, the WRITE DELAY is set to 0 seconds.
19     */
20     private final static String HSQL_ADDITIONAL_PARAMS = ";hsqldb.write_delay=false;sql.syntax_pgs=true";
21     private final static String DB_URL = String.format("jdbc:hsqldb:file:%s/%s", DB_DIR, DB_NAME, HSQL_ADDITIONAL_PARAMS);
22
23
24     private final static HikariDataSource ds;
25
26     static {
27         HikariConfig config = new HikariConfig();
28         config.setJdbcUrl(DB_URL);
29         config.setUsername(DB_USERNAME);
30         config.setPassword(DB_USER_PASSWORD);
31         config.setMaximumPoolSize(1);
32         ds = new HikariDataSource(config);
33     }
34
35     private DataSource() {}
36
37     public static Connection getConnection() throws SQLException {
38         return ds.getConnection();
39     }
40
41 }
```

Screen 4 - klasa DataSource.java realizująca pooling połączeń poprzez HikariCP

2.5. Inicjalizacja bazy danych oraz transakcje w klasie DbInitializer

Klasa DbInitializer posiada statyczną tablicę kwerend DDL, które tworzą tabele projekt oraz zadanie, odpowiednie klucze główne, unikalne indeksy na wyszukiwane kolumny (np. `CREATE INDEX IF NOT EXISTS projekt_nazwa_idx ON projekt (nazwa)`) oraz definiują relacje kluczy obcych.

Celem implementacji metody `init()` w tej klasie było zapewnienie pełnej transakcyjności (ACID). Przed przystąpieniem do wykonywania zapytań, tryb automatycznego zatwierdzania jest wyłączany (`connection.setAutoCommit(false)`). Kwerendy są przetwarzane w pętli wewnątrz bloku `try`. Jeżeli wszystkie polecenia zakończą się sukcesem, transakcja jest zatwierdzana za pomocą `connection.commit()`. W przypadku wystąpienia choćby jednego błędu `SQLException` (np. błąd składni, naruszenie integralności struktury), w bloku `catch` wywoływana jest metoda `connection.rollback()`, wycofująca wszystkie dotychczasowe operacje. Gwarantuje to spójność bazy i zapobiega częściowemu utworzeniu tabel. Blok `finally` dba o przywrócenie pierwotnego stanu `autocommit` dla połączenia.

```

DbInitializer.java x
1 package com.project.datasource;
2
3 import java.sql.Connection;
4 import java.sql.SQLException;
5 import java.sql.Statement;
6 import org.slf4j.Logger;
7 import org.slf4j.LoggerFactory;
8
9 public class DbInitializer {
10     private static final Logger logger = LoggerFactory.getLogger(DbInitializer.class);
11     private static final String[] queries = {
12         ""
13         CREATE TABLE IF NOT EXISTS projekt(
14             projekt_id SERIAL, --SERIAL lub INTEGER GENERATED BY DEFAULT AS IDENTITY(START WITH 1, INCREMENT BY 1)
15             nazwa VARCHAR(50) NOT NULL,
16             opis VARCHAR(1000),
17             dataczas_utworzenia TIMESTAMP DEFAULT now(),
18             data_oddania DATE,
19             CONSTRAINT projekt_pk PRIMARY KEY (projekt_id)
20         );
21         CREATE TABLE IF NOT EXISTS zadanie(
22             zadanie_id SERIAL, --SERIAL lub INTEGER GENERATED BY DEFAULT AS IDENTITY(START WITH 1, INCREMENT BY 1)
23             nazwa VARCHAR(50) NOT NULL,
24             opis VARCHAR(1000),
25             kolejnosc INTEGER,
26             dataczas_utworzenia TIMESTAMP DEFAULT now(),
27             projekt_id INTEGER NOT NULL,
28             CONSTRAINT zadanie_pk PRIMARY KEY (zadanie_id)
29         );
30         ""
31         ""
32         CREATE INDEX IF NOT EXISTS projekt_nazwa_idx ON projekt(nazwa);
33         CREATE INDEX IF NOT EXISTS zadanie_nazwa_idx ON zadanie(nazwa);
34         ALTER TABLE zadanie ADD CONSTRAINT IF NOT EXISTS zadanie_projekt_fk FOREIGN KEY (projekt_id) REFERENCES projekt (projekt_id) ON
35         ALTER TABLE zadanie ADD CONSTRAINT IF NOT EXISTS unique_kolejnosc UNIQUE (kolejnosc, projekt_id);
36         ""
37     };
38
39     private DbInitializer() {}
40
41     public static void init() {
42         try (Connection connection = DataSource.getConnection()) {
43             boolean initialAutocommit = connection.getAutoCommit();
44             connection.setAutoCommit(false);
45             try (Statement stmt = connection.createStatement()) {
46                 for (int i = 0; i < queries.length; i++) {
47                     stmt.executeUpdate(queries[i]);
48                     logger.info("QUERY {}: \n{}", i + 1, queries[i]);
49                 }
50                 connection.commit();
51             } catch (SQLException e) {
52                 connection.rollback();
53                 throw new RuntimeException(e);
54             } finally {
55                 if (initialAutocommit)
56                     connection.setAutoCommit(true);
57             }
58         } catch (SQLException e) {
59             throw new RuntimeException(e);
60         }
61     }
62 }

```

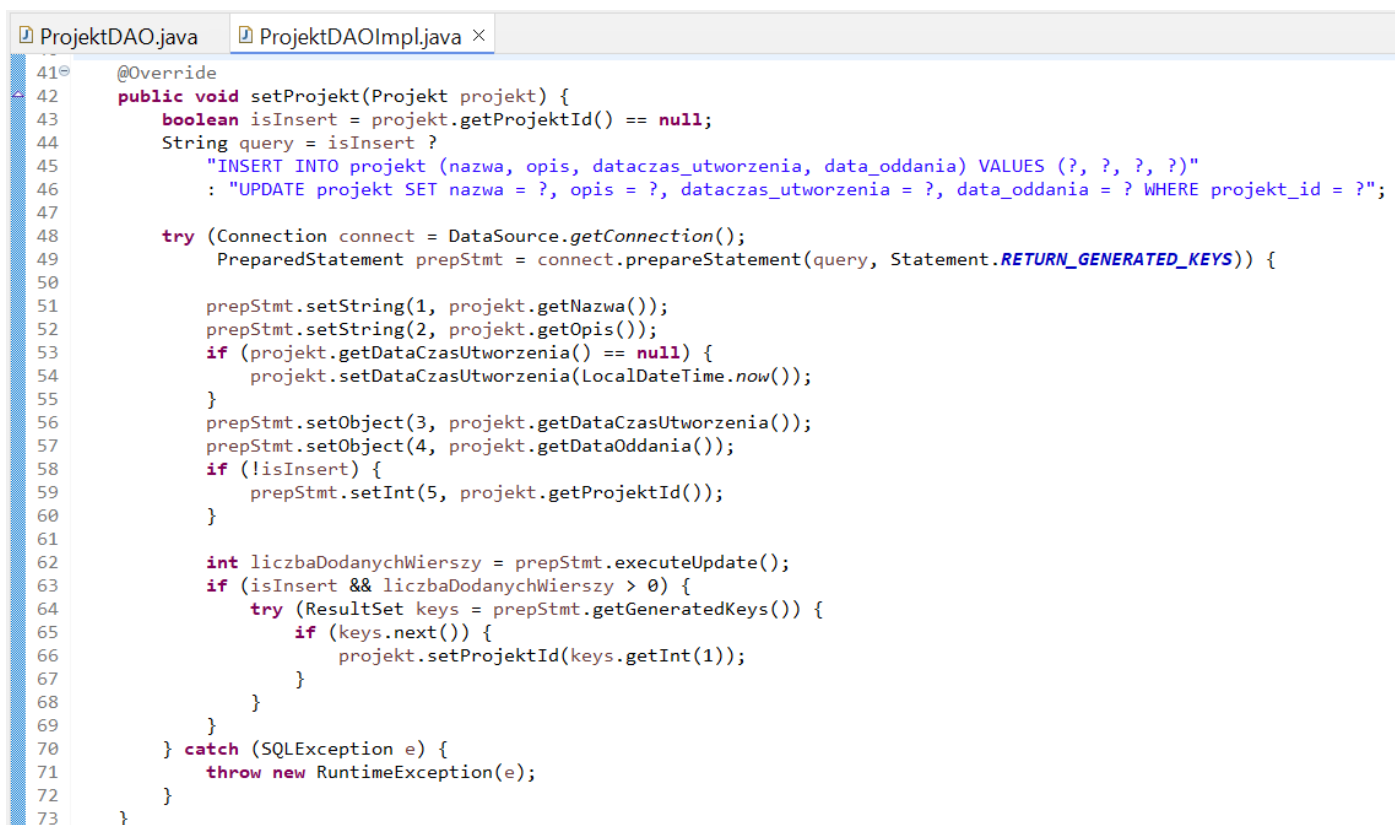
Screen 5 - zaznaczona czerwonym prostokątem metoda init() w klasie DbInitializer.java z obsługą transakcji commit/rollback

2.6. Warstwa dostępu do danych - interfejsy i implementacje DAO (ProjektDAO, ProjektDAOImpl, ZadanieDAO, ZadanieDAOImpl)

Warstwa DAO zapewnia abstrakcję i separację logiki SQL od warstwy prezentacji interfejsu graficznego. Interfejs ProjektDAO definiuje kontrakt dla klasy implementującej, zawierając sygnatury metod takich jak: pobieranie pojedynczego projektu, zapis/aktualizacja (setProjekt), usuwanie, pobieranie list z paginacją oraz zliczanie wierszy.

Klasa ProjektDAOImpl realizuje te operacje za pomocą zapytań JDBC. W metodzie setProjekt, na podstawie wartości pola projektId (równe null lub posiadające wartość) podejmowana jest decyzja: czy wygenerować instrukcję INSERT, czy UPDATE.

Podczas dodawania nowego projektu wykorzystywany jest mechanizm Statement.RETURN_GENERATED_KEYS, który pozwala natychmiastowo pobrać unikalny identyfikator wygenerowany automatycznie przez bazę danych i przypisać go z powrotem do przekazanego obiektu Javy. Z kolei zapytania filtrujące implementują stronicowanie bezpośrednio na poziomie bazy danych przy użyciu klauzul OFFSET oraz LIMIT.



```
41 @Override
42 public void setProjekt(Projekt projekt) {
43     boolean isInsert = projekt.getProjektId() == null;
44     String query = isInsert ?
45         "INSERT INTO projekt (nazwa, opis, dataczas_utworzenia, data_oddania) VALUES (?, ?, ?, ?)"
46         : "UPDATE projekt SET nazwa = ?, opis = ?, dataczas_utworzenia = ?, data_oddania = ? WHERE projekt_id = ?";
47
48     try (Connection connect = DataSource.getConnection();
49         PreparedStatement prepStmt = connect.prepareStatement(query, Statement.RETURN_GENERATED_KEYS)) {
50
51         prepStmt.setString(1, projekt.getNazwa());
52         prepStmt.setString(2, projekt.getOpis());
53         if (projekt.getDataCzasUtworzenia() == null) {
54             projekt.setDataCzasUtworzenia(LocalDate.now());
55         }
56         prepStmt.setObject(3, projekt.getDataCzasUtworzenia());
57         prepStmt.setObject(4, projekt.getDataOddania());
58         if (!isInsert) {
59             prepStmt.setInt(5, projekt.getProjektId());
60         }
61
62         int liczbaDodanychWierszy = prepStmt.executeUpdate();
63         if (isInsert && liczbaDodanychWierszy > 0) {
64             try (ResultSet keys = prepStmt.getGeneratedKeys()) {
65                 if (keys.next()) {
66                     projekt.setProjektId(keys.getInt(1));
67                 }
68             }
69         }
70     } catch (SQLException e) {
71         throw new RuntimeException(e);
72     }
73 }
```

Screen 6 - Implementacja metody setProjekt oraz mechanizmu pobierania wygenerowanych kluczy głównych w pliku ProjektDAOImpl.java

2.7. Konfiguracja mechanizmu rejestracji w pliku logback.xml

W celu monitorowania stanu aplikacji oraz diagnostyki błędów, zamieniłem `System.out.println()` na mechanizm rejestracji zdarzeń oparty na SLF4J. Konfiguracja zawarta w pliku `logback.xml` definiuje dwa niezależne ujścia (Appenders) strumieni logowania.

Pierwszy z nich to `STDOUT` (`ConsoleAppender`), który wypisuje komunikaty w konsoli Eclipse. Drugi to `FILE` (`RollingFileAppender`), który kieruje logi do pliku tekstowego w katalogu `logs`. Zastosowanie `TimeBasedRollingPolicy` zapewnia automatyczną rotację plików logów każdego dnia. Stare logi są kompresowane do archiwum ZIP. Skonfigurowałem też limity przetrzymywania: maksymalne przechowywanie historii przez 30 dni oraz całkowity limit rozmiaru archiwum ustawiony na 30MB, co zapobiega niekontrolowanemu zapełnieniu dysku twardego.

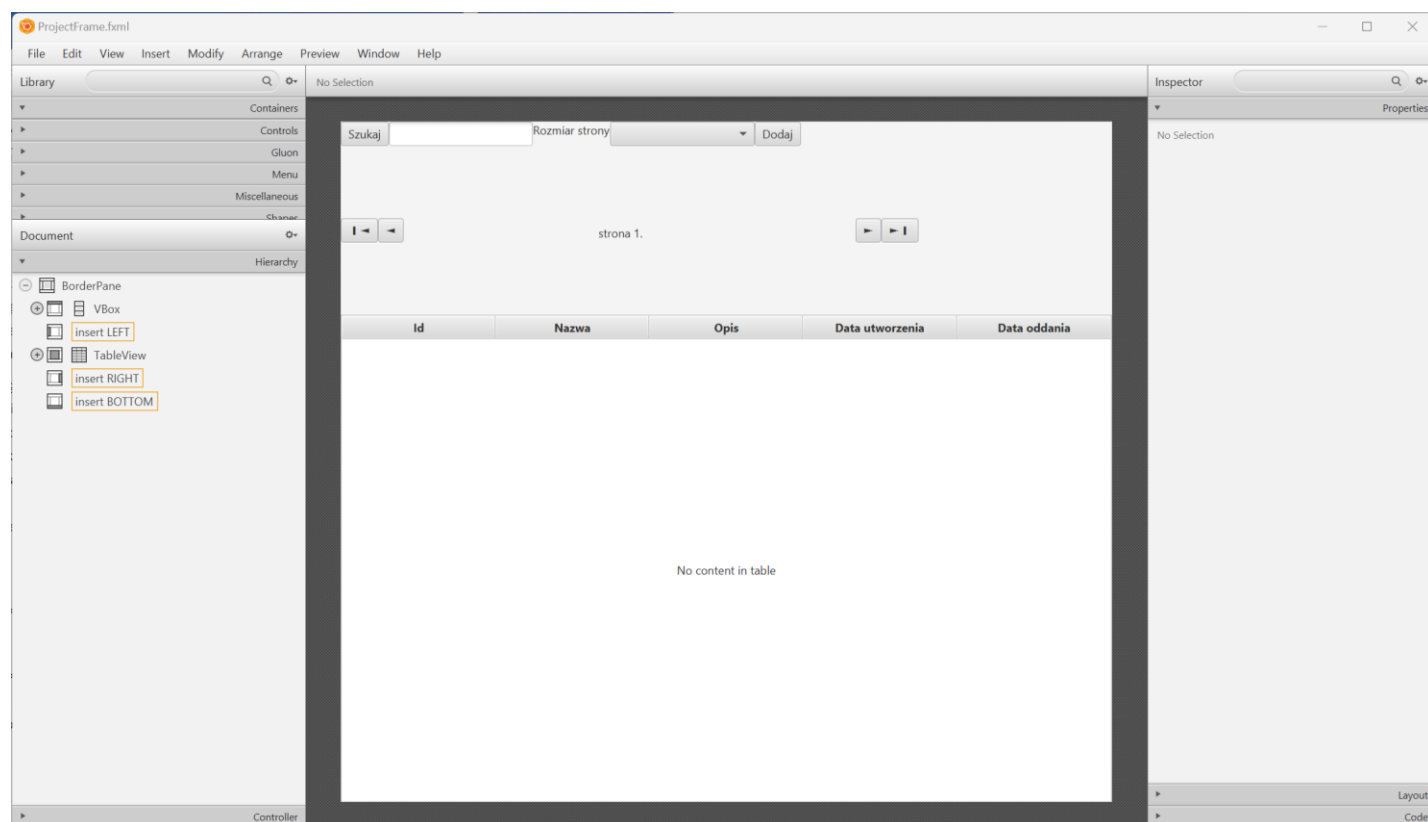
```
logback.xml x
1 <?xml version="1.0" encoding="UTF-8"?>
  Bind to grammar/schema...
2 <configuration debug="true">
3   <property name="LOG_FILE" value="project-jfx-client" />
4   <property name="LOG_DIR" value="logs" />
5   <property name="LOG_ARCHIVE" value="${LOG_DIR}/archive" />
6
7   <!-- Send messages to System.out -->
8   <appender name="STDOUT">
9     <class>"ch.qos.logback.core.ConsoleAppender">
10    <encoder>
11      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36}.%M\(%line\) - %msg%n</pattern>
12    </encoder>
13  </appender>
14
15  <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
16    <file>${LOG_DIR}/${LOG_FILE}.log</file>
17    <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
18      <!-- daily rollover -->
19      <fileNamePattern>${LOG_ARCHIVE}/%d{yyyy-MM-dd}${LOG_FILE}.log.zip
20    </fileNamePattern>
21    <!-- keep 30 days' worth of history capped at 30MB total size -->
22    <maxHistory>30</maxHistory>
23    <totalSizeCap>30MB</totalSizeCap>
24  </rollingPolicy>
25  <encoder>
26    <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36}.%M\(%line\) - %msg%n</pattern>
27  </encoder>
28 </appender>
29
30 <logger name="com.project" level="INFO" additivity="false">
31   <appender-ref ref="STDOUT" />
32   <appender-ref ref="FILE" />
33 </logger>
34
35 <!-- By default, the level of the root level is set to DEBUG -->
36 <root level="INFO">
37   <appender-ref ref="STDOUT" />
38 </root>
39 </configuration>
```

Screen 7 - struktura pliku konfiguracyjnego `logback.xml` ze zdefiniowaną polityką rotacji plików

2.8. Graficzny interfejs użytkownika - plik FXML, Scene Builder i styl CSS

Interfejs użytkownika został zaprojektowany w taki sposób, by podstawowym kontenerem całego widoku był `BorderPane`. W jego górnej sekcji (TOP) umieściłem panele `VBox` i `HBox`, w których znajdują się: pole tekstowe do wprowadzania fraz wyszukiwania, przycisk Szukaj, rozwijana lista wyboru liczby rekordów na stronę (`ChoiceBox`) oraz przycisk Dodaj projekt.

Poniżej paska narzędziowego jest komponent `GridPane`, w którym są symetrycznie rozmieszczone przyciski nawigacyjne stronicowania oraz środkowa etykieta informująca o bieżącym numerze strony (np. strona 1). W części centralnej (CENTER) znajduje się główny komponent `TableView<Projekt>`, posiadający jawnie zdefiniowane kolumny dla każdego pola modelu. Wygląd tabeli wzbogaciłem za pomocą arkusza stylów `application.css`. Wprowadziłem w nim regułę alternatywnego kolorowania wierszy (`.table-row-cell:odd`), która ułatwia czytanie danych, oraz style podświetlenia zaznaczonego wiersza przy użyciu systemowych akcentów kolorów.



Screen 8 - wygląd zaprojektowanego interfejsu użytkownika w programie Scene Builder

2.9. Kontroler aplikacji, czyli klasa ProjectController, ObservableList i wielowątkowość

Klasa ProjectController łączy warstwę wizualną FXML z logiką biznesową. Komponenty GUI są automatycznie wstrzykiwane przez JavaFX do pól kontrolera oznaczonych adnotacją @FXML. Powiązanie danych w tabeli z obiektami Javy zrealizowałem za pomocą właściwości (Properties) oraz klas fabryk PropertyValueFactory, które mapują nazwy pól modelu na odpowiednie kolumny tabeli w metodzie initialize().

Poważnym wyzwaniem projektowym była implementacja asynchronicznego przetwarzania wielowątkowego. W związku z tym, że operacje JDBC blokują wątek, w którym są wywoływane, wykonanie ich w głównym wątku aplikacji JavaFX spowodowałoby całkowite zamrożenie interfejsu podczas dłuższego zapytania SQL. Aby temu zapobiec, kontroler inicjalizuje jednowątkową pulę ExecutorService (Executors.newFixedThreadPool(1)). Każde żądanie załadowania danych (metoda loadPage) jest opakowywane w zadanie typu Runnable i przekazywane do wykonawcy. Po pobraniu danych z bazy, aktualizacja listy ObservableList (projekty.clear() oraz projekty.addAll()) jest bezpiecznie oddelegowywana do wątku interfejsu za pomocą asynchronicznej metody Platform.runLater().

```

ProjectController.java ×
254 private void loadPage(String search4, Integer pageNo, Integer pageSize) {
255     try {
256         final List<Projekt> projektList = new ArrayList<>();
257         if (search4 != null && !search4.isEmpty()) {
258             if (search4.matches("[0-9]+")) {
259                 // Identyfikator
260                 Integer id = Integer.parseInt(search4);
261                 Projekt p = projektDAO.getProjekt(id);
262                 if (p != null) {
263                     projektList.add(p);
264                 }
265             } else if (search4.matches("^([0-9]{4})-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])$")) {
266                 // Data
267                 LocalDate data = LocalDate.parse(search4);
268                 projektList.addAll(projektDAO.getProjektyWhereDataOddaniaIs(data, pageNo * pageSize, pageSize));
269             } else {
270                 // Nazwa
271                 projektList.addAll(projektDAO.getProjektyWhereNazwaLike(search4, pageNo * pageSize, pageSize));
272             }
273         } else {
274             // Brak filtrów - Pobierz wszystkie
275             projektList.addAll(projektDAO.getProjekty(pageNo * pageSize, pageSize));
276         }
277
278         Platform.runLater(() -> {
279             projekty.clear();
280             projekty.addAll(projektList);
281         });
282     } catch (RuntimeException e) {
283         String errMsg = "Błąd podczas pobierania listy projektów.";
284         Logger.error(errMsg, e);
285         String errDetails = e.getCause() != null ?
286             e.getMessage() + "\n" + e.getCause().getMessage() : e.getMessage();
287         Platform.runLater(() -> showError(errMsg, errDetails));
288     }
289 }

```

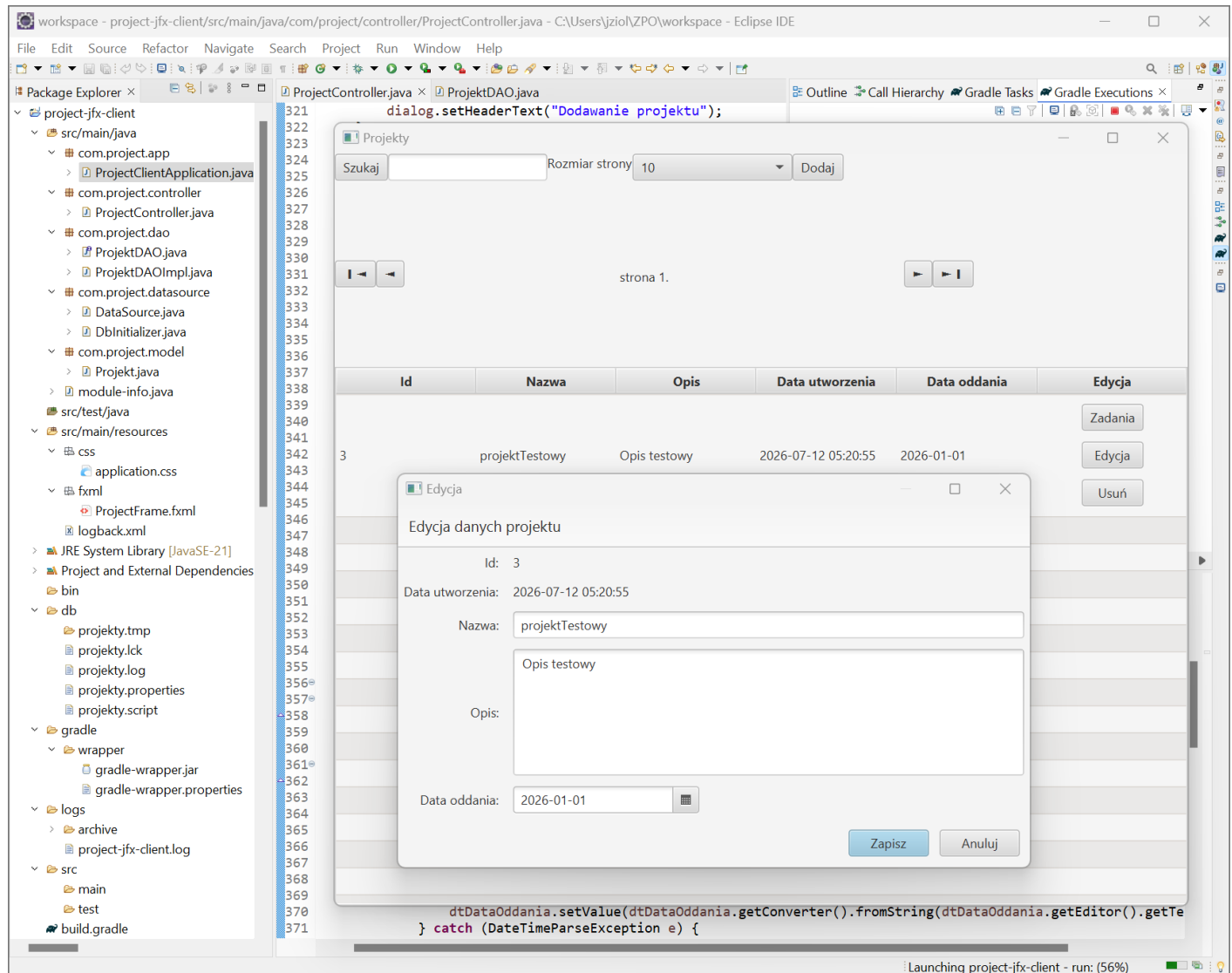
Screen 9 - kod źródłowy mechanizmu wielowątkowego loadPage() wykorzystującego Platform.runLater()

2.10. Operacje CRUD w interfejsie - dodawanie, edycja i usuwanie projektów

Operacje dodawania oraz modyfikacji danych projektów zostały zawarte w jednej metodzie prywatnej edytujProjekt. Metoda ta dynamicznie generuje okno dialogowe klasy Dialog<Projekt>. Wnętrze okna jest zbudowane za pomocą układu GridPane, w którym osadzone są etykiety oraz komponenty wejściowe (TextField dla nazwy, TextArea dla opisu oraz DatePicker dla terminu oddania).

Dla kontrolki DatePicker niestandardowy StringConverter dba o poprawną konwersję tekstu na obiekt LocalDate i odwrotnie, eliminując błędy rzucania wyjątków w przypadku ręcznego wpisania daty przez użytkownika. Po wciśnięciu przycisku 'Zapisz', dane są weryfikowane, a obiekt trafia do bazy danych przez wątek. Po udanym zapisie tabela jest natychmiastowo odświeżana. Usuwanie rekordów powiązałem z przyciskiem Usuń osadzonym w kolumnie akcji

tabeli. Wywołuje ono okno potwierdzenia Alert (AlertType.CONFIRMATION). Po kliknięciu 'OK' przez użytkownika, rekord jest kasowany z bazy, a kaskadowe powiązania ON DELETE CASCADE automatycznie usuwają zależne zadania.



Screen 10 - na tle okna programu Eclipse i okna Projekty, widoczne okno dialogowe dedykowane dla edytowania danych projektu

2.11. Mechanizm wyszukiwania i stronicowania danych

Stronicowanie (paginacja) danych działa w oparciu o wyliczanie przesunięcia (offset) przekazywanego do zapytania SQL na podstawie wzoru: $\text{offset} = \text{pageNo} * \text{pageSize}$. Zmiana pozycji w ChoiceBox powoduje natychmiast zaktualizowanie rozmiaru strony i reset licznika stron do zera. Przyciski 'Dalej' i 'Wstecz' inkrementują lub dekrementują zmienną pageNo, sprawdzając uprzednio warunki (całkowitą liczbę wierszy w bazie), zapobiegając wyjściu poza zakres dostępnych danych.

W mechanizmie wyszukiwania, w metodzie loadPage zawarte są instrukcje warunkowe testujące zawartość pola tekstowego txtSzukaj przy użyciu wyrażeń regularnych. Jeżeli wpisana fraza dopasuje się do wzorca numerycznego ([0-9]+), aplikacja założy, że użytkownik poszukuje konkretnego projektu po jego unikalnym kluczu głównym (Id) i wywoła dedykowaną metodę DAO getProjekt(). Jeżeli tekst odpowiada strukturze daty (RRRR-MM-DD), dane są filtrowane po terminie oddania. W pozostałych przypadkach wyszukiwanie realizowane jest tekstowo po nazwie projektu przy użyciu klauzuli LIKE %szukany_tekst%.

2.12. Tworzenie wersji uruchomieniowej (jlink i jpackage)

Aplikacja została zmodularyzowana według Java Platform Module System (JPMS). Dlatego w głównym katalogu projektu utworzyłem plik module-info.java. Definiuje on nazwę modułu project.jfx.client oraz deklaruje exports (udostępnianie pakietów na zewnątrz), jak i requires (wymagania modułów zewnętrznych JavaFX oraz HikariCP).

Niezwykle istotne było dodanie instrukcji opens ... to javafx.fxml, javafx.graphics, javafx.base. Bez jawnego otwarcia tych pakietów, JavaFX nie miałby uprawnień refleksji do wstrzykiwania komponentów GUI do pól kontrolera oraz odczytywania właściwości metod get/set w klasach modelowych, co skutkowałoby błędem IllegalAccessException podczas uruchomienia.

Po poprawnej konfiguracji modułów, wywołałem zadanie Gradle jpackageImage. Narzędzie jlink wygenerowało zoptymalizowany obraz JRE, a narzędzie jpackage utworzyło niezależny plik wykonywalny .exe, gotowy do dystrybucji.

```

1 module project.jfx.client {
2     exports com.project.datasource;
3     exports com.project.dao;
4     exports com.project.model;
5     exports com.project.app;
6     exports com.project.controller;
7
8     requires javafx.base;
9     requires javafx.fxml;
10    requires javafx.controls;
11    requires transitive javafx.graphics;
12    requires com.zaxxer.hikari;
13    requires transitive java.sql;
14    requires org.hsqldb;
15    requires org.slf4j;
16    requires ch.qos.logback.classic;
17    requires ch.qos.logback.core;
18
19    opens com.project.app to javafx.graphics, javafx.fxml, javafx.base, javafx.controls;
20    opens com.project.model to javafx.graphics, javafx.fxml, javafx.base, javafx.controls;
21    opens com.project.controller to javafx.graphics, javafx.fxml, javafx.base, javafx.controls;
22 }

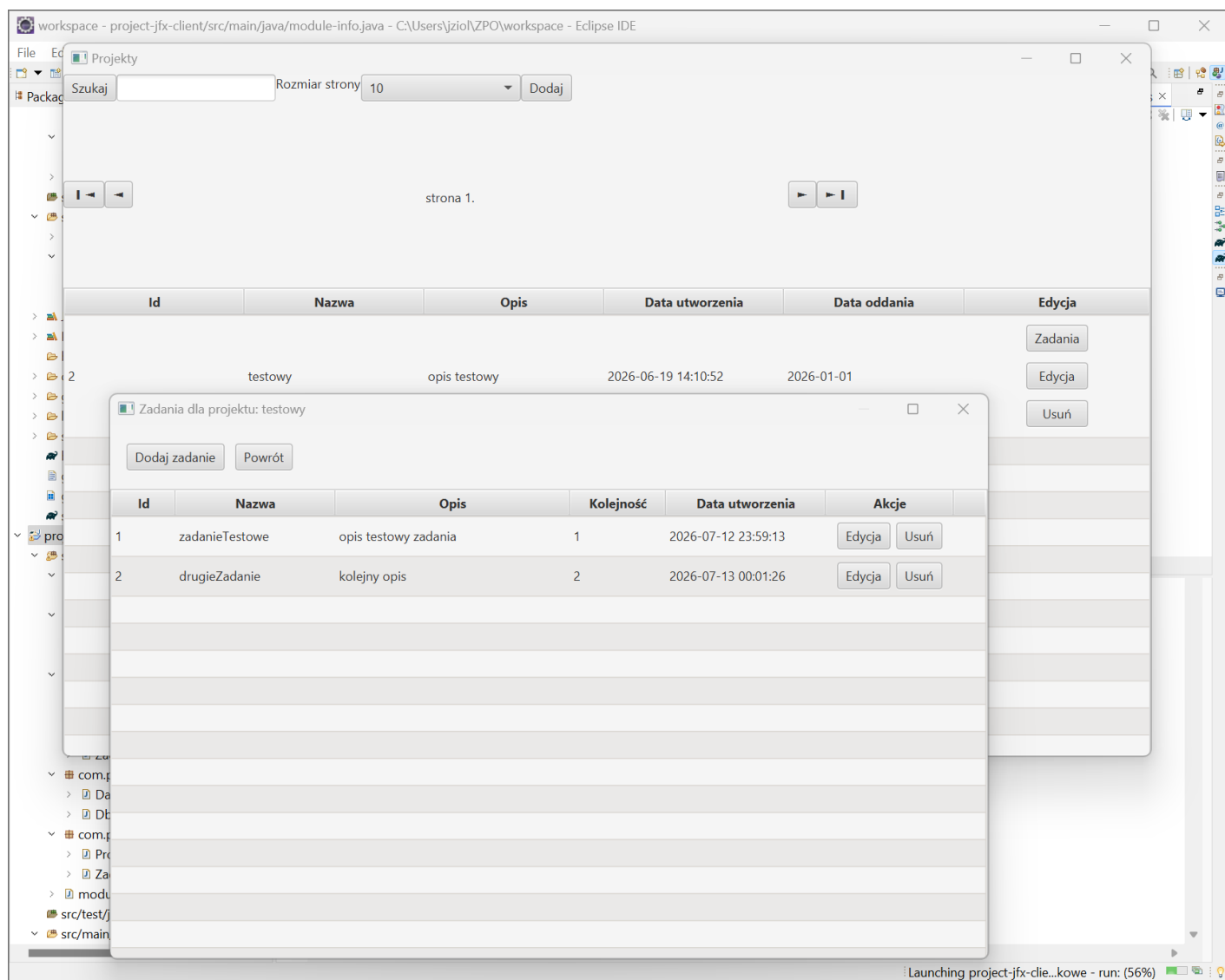
```

Screen 11 - zawartość pliku konfiguracyjnego module-info.java z dyrektywami exports, requires oraz opens

2.13. Realizacja zadania dodatkowego

W ramach zadania dodatkowego zaimplementowałem mechanizmy zarządzania zadaniami przypisanymi do wybranego projektu, co rozwinęło relację jeden-do-wielu (1:N) na poziomie bazy danych oraz interfejsu użytkownika. W tym celu utworzyłem nową encję bazodanową tabeli zadanie oraz zaprojektowałem odrębny widok graficzny w pliku ZadanieFrame.fxml, zawierający tabelę zadań oraz przyciski nawigacyjne.

Logika tego okna podrzędnego jest zarządzana przez nowo utworzoną klasę `com.project.controller.ZadanieController`. Kliknięcie przycisku „Zadania” w dowolnym wierszu głównej tabeli projektów przechwytuje obiekt zaznaczonego projektu i przekazuje go do metody `openZadanieFrame()`. Okno zadań otwierane jest jako okno modalne (`stage.initModality(Modality.APPLICATION_MODAL)`). To blokuje możliwość klikania w okno główne i wymusza na użytkowniku zakończenie operacji na zadaniach przed powrotem do menu głównego. Fabryka kontrolerów `loader.setControllerFactory()` pozwala przekazać współdzieloną pulę wątków `ExecutorService` oraz instancję DAO, dzięki czemu możliwe jest zachowanie klarowności wstrzykiwania zależności.



Screen 12 - widok aplikacji z uruchomionym modalnym oknem zarządzania zadaniami dla wybranego projektu

3. Wnioski końcowe/Podsumowanie

Trzecie laboratorium z przedmiotu Zaawansowane Programowanie Obiektowe łączyło zagadnienia warstwy persistencji danych, wzorców projektowych, wielowątkowości oraz budowy modularnych interfejsów graficznych.

Najważniejszym wnioskiem płynącym z realizacji zadania jest konieczność separacji operacji wejścia/wyjścia od głównego wątku obsługi interfejsu użytkownika. Zastosowanie puli wątków `ExecutorService` w połączeniu z `Platform.runLater()` bardzo poprawiło stopień responsywności aplikacji i pozwoliło zachować płynność podczas wykonywania operacji bazodanowych.

Zastosowanie wzorców projektowych takich jak `Singleton` (w klasie `DataSource`) oraz `Data Access Object (DAO)` zapewniło niezwykle ważne praktyczne zalety. Dzięki odizolowaniu zapytań SQL w klasie implementacji DAO, kod kontrolera interfejsu nie zajmują niskopoziomowe instrukcje JDBC.

Natomiast zrealizowanie zadania dodatkowego obejmowało utworzenie relacji bazodanowych oraz dynamiczne wstrzykiwanie zależności poprzez fabryki kontrolerów frameworka JavaFX. Z kolei narzędzia `jlink` i `jpackage` w pewien sposób przybliżyły sposób, w jaki oprogramowanie desktopowe jest przygotowywane do dystrybucji w profesjonalnych zastosowaniach.

Podsumowując, cel laboratorium został w pełni osiągnięty. Zaprojektowana aplikacja stanowi wydajną i elastyczną podstawę do dalszego rozwoju podobnych stabilnych systemów informatycznych.